

Análisis de Decisiones

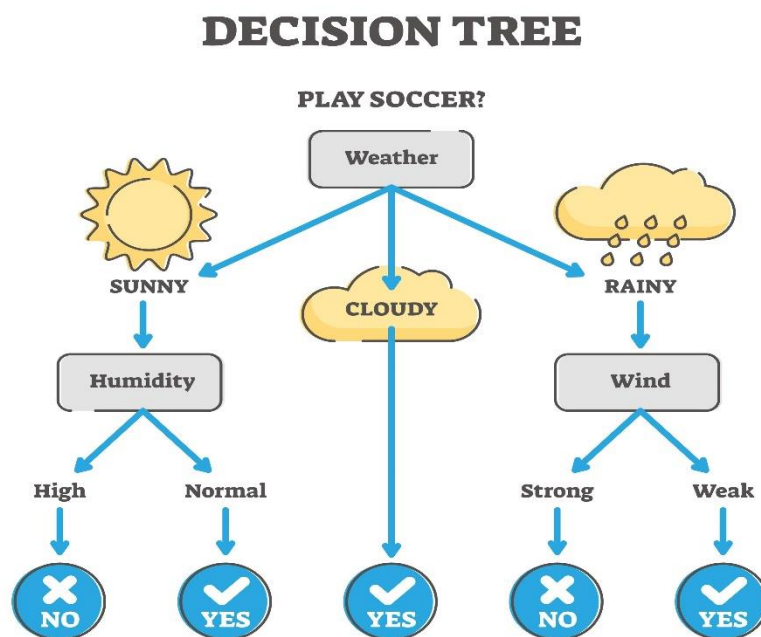
El **Análisis de Decisiones** es un enfoque estructurado y sistemático para tomar decisiones complejas bajo incertidumbre. Consiste en descomponer una decisión grande en sus partes constituyentes: **alternativas** (opciones), **consecuencias** (resultados), e **incertidumbre** (eventos futuros desconocidos) para elegir el curso de acción óptimo.

Componentes del Análisis de Decisiones

El proceso de Análisis de Decisiones se basa en los siguientes elementos clave:

1. **Alternativas:** Las diferentes acciones o estrategias que el tomador de decisiones puede elegir.
2. **Estados de la Naturaleza (Incetidumbre):** Los eventos futuros que están fuera del control del tomador de decisiones y que afectarán el resultado.
3. **Resultados (Payoffs):** El valor o utilidad (beneficio, costo, tiempo, etc.) asociado a cada combinación de una alternativa y un estado de la naturaleza.
4. **Probabilidades:** Las estimaciones de la ocurrencia de cada estado de la naturaleza.
5. **Criterio de Decisión:** La regla matemática utilizada para seleccionar la mejor alternativa (e.g., maximizar el valor esperado).

Las herramientas comunes para aplicar este análisis incluyen las **Matrices de Decisión** y, más a menudo, los **Árboles de Decisión**.



Aplicación en la Ingeniería

En la Ingeniería, el Análisis de Decisiones es fundamental para gestionar proyectos, arquitectura de sistemas y riesgos, ya que casi todas las decisiones implican compensaciones (trade-offs) entre tiempo, costo y calidad.

Área de Aplicación	Problema de Decisión	Alternativas	Criterio de Decisión Típico
Arquitectura de Software	¿Qué base de datos utilizar?	PostgreSQL vs. MongoDB vs. MySQL	Minimizar Latencia y Costo (Maximizar Valor Esperado).
Gestión de Proyectos	¿Deberíamos retrasar el lanzamiento para corregir un <i>bug</i> de riesgo medio?	Lanzar ahora vs. Retrasar una semana.	Maximizar la Satisfacción del Cliente (minimizar el riesgo de fallo post-lanzamiento).
Ciberseguridad	¿Cuánto invertir en nuevos sistemas de <i>firewall</i> ?	Invertir poco vs. Invertir moderado vs. Invertir mucho.	Minimizar el Costo Esperado de las Brechas de Seguridad.
Desarrollo de IA/ML	¿Qué algoritmo usar para la predicción de precios?	Regresión Lineal vs. Red Neuronal vs. Árboles de Impulso (Boosting).	Maximizar la Precisión (<i>Accuracy</i>) del modelo.

La Inteligencia Artificial (IA) y la Toma de Decisiones

Sí, la IA no solo puede, sino que se está convirtiendo en la herramienta **más poderosa** para asistir y, en algunos casos, automatizar la toma de decisiones.

La IA ayuda en dos funciones principales dentro del proceso de decisión:

1. Cuantificación de la Incertidumbre (Función Analítica)

- **Cálculo de Probabilidades:** Los modelos de **Machine Learning (ML)** y **Deep Learning** analizan grandes volúmenes de datos históricos para estimar las probabilidades de los Estados de la Naturaleza. Por ejemplo, un modelo de ML puede predecir la probabilidad de que un servidor falle en el próximo mes.
- **Detección de Patrones:** Los sistemas de IA detectan correlaciones complejas que son invisibles para los humanos, lo que permite identificar nuevas alternativas o riesgos no considerados.

2. Automatización y Optimización (Función Prescriptiva)

- **Optimización de Búsqueda:** Algoritmos como la **Optimización por Enjambre (Swarm Optimization)** o el **Aprendizaje por Refuerzo (Reinforcement Learning)** pueden simular millones de escenarios en un **Árbol de Decisión** para encontrar la política óptima que maximice la recompensa esperada.
- **Sistemas Expertos:** En áreas bien definidas (como el diagnóstico médico o la detección de *spam*), la IA puede tomar decisiones de bajo nivel de manera autónoma, adhiriéndose a un conjunto de reglas predefinidas y actualizables.
- **Motores de Recomendación:** Estos sistemas toman la decisión diaria de qué producto o contenido mostrar a un usuario individual, maximizando la probabilidad de una interacción deseada.

Aplicaciones

El **Análisis de Decisiones** se aplica en la Ciencia de Datos para **estructurar la incertidumbre** y **cuantificar el valor esperado** de las acciones basadas en predicciones o modelos. En esencia, convierte la **precisión predictiva** de un modelo de Machine Learning en **valor económico** o **utilidad empresarial**.

Aquí tienes 5 casos clave que permiten comprender cómo se aplica en el ámbito de la Ciencia de Datos:

Casos de Análisis de Decisiones en Ciencia de Datos

1. Clasificación con Costos Asimétricos (Umbral de Decisión)

Concepto	Aplicación	Análisis de Decisiones
Problema: Clasificar una instancia como positiva o negativa, donde los costos de error no son iguales (e.g., el costo de un Falso Negativo es mucho mayor que el de un Falso Positivo).	Caso: Detección de fraude con tarjeta de crédito.	La Ciencia de Datos predice la probabilidad de fraude. El Análisis de Decisiones utiliza un árbol de decisión y una matriz de costos para determinar el <i>umbral de probabilidad</i> óptimo para clasificar como "fraude" (acción: bloquear la transacción). Se minimiza el Costo Esperado Total (pérdida por fraude no detectado + pérdida por bloqueo erróneo).

2. Optimización del Despliegue de Recursos (Mantenimiento Predictivo)

Concepto	Aplicación	Análisis de Decisiones
Problema: Decidir cuándo programar el mantenimiento de un equipo industrial para evitar fallas costosas.	Caso: Mantenimiento predictivo de servidores o turbinas.	Un modelo de ML predice la Probabilidad de Falla en el próximo mes. El árbol de decisión evalúa dos alternativas: a) Mantenimiento preventivo ahora (costo bajo) o b) No hacer nada (costo potencial alto si falla, o costo cero si no falla). La decisión es la que minimiza el Costo Esperado (CE) .

3. Asignación de Presupuesto en Marketing (Ofertas Personalizadas)

Concepto	Aplicación	Análisis de Decisiones
Problema: Decidir si ofrecer un descuento costoso a un cliente o no, basándose en la probabilidad de que realice una compra.	Caso: Campañas de <i>retention</i> (retención de clientes).	Un modelo de ML predice la Propensión de Compra del Cliente A. El Análisis de Decisiones utiliza el árbol para sopesar la Ganancia Esperada (VME) : (Probabilidad de compra con descuento $\times$ Ganancia Neta) vs. (Probabilidad de compra sin descuento $\times$ Ganancia Neta original). La decisión es la que maximiza el VME .

4. Selección de Algoritmos bajo Riesgo (Rendimiento del Modelo)

Concepto	Aplicación	Análisis de Decisiones
Problema: Elegir entre dos o más modelos (algoritmos) para la producción, donde cada modelo tiene diferentes requisitos de implementación y precisión.	Caso: Implementación de un modelo de predicción de la demanda.	Alternativas: Modelo A (sencillo de implementar, precisión media) vs. Modelo B (complejo de implementar, precisión alta). Los "Estados de la Naturaleza" pueden ser: "La precisión alta es necesaria" (Prob. p_1) o "La precisión media es suficiente" (Prob. p_2). La matriz de resultados mide el Beneficio Neto (Beneficio por precisión - Costo de implementación). Se elige el modelo con el VME más alto .

5. Gestión de Proyectos (Despliegue de Producto)

Concepto	Aplicación	Análisis de Decisiones
Problema: Decidir si lanzar un producto o sistema ahora con <i>bugs</i> conocidos o retrasar el lanzamiento para corregirlos.	Caso: <i>Software</i> con riesgo residual.	Alternativas: Lanzar Ahora o Retrasar 2 Semanas . Los estados de la naturaleza: a) Falla Crítica Post-lanzamiento (Prob. p_1) o b) No Falla (Prob. p_2). El modelo de ML ayuda a estimar p_1 (Probabilidad de Falla Crítica). La matriz de resultados evalúa el Costo de Oportunidad del retraso vs. el Costo Esperado de la Falla post-lanzamiento. Se elige la alternativa que minimiza el CE .

A continuación, desarrollaremos 4 ejemplos básicos de análisis de decisiones desarrollados paso a paso con su respectivo código en RStudio, utilizando la estructura de **Valor Monetario Esperado (VME)**, que es el criterio de decisión más común.

1. Ejemplo: Decisión sobre la Tecnología de Base de Datos

Escenario

Una empresa debe elegir entre dos bases de datos para un nuevo sistema: **PostgreSQL (segura y estable)** o **MongoDB (rápida, pero con riesgo de curva de aprendizaje)**. El resultado (ganancia) depende de si la demanda del proyecto resulta ser **Alta** o **Baja**.

Matriz de Resultados (Ganancia Neta, en miles de USD)

Alternativa	Demanda Alta (Prob=0.6)	Demanda Baja (Prob=0.4)
A1: PostgreSQL	150	70
A2: MongoDB	200	40

Exportar a Hojas de cálculo

Análisis de Decisiones con RStudio

Paso 1: Definir la Matriz y Probabilidades

```
# Definir los resultados (matriz de ganancias)
resultados <- matrix(c(
  150, 70, # PostgreSQL: Alta, Baja
  200, 40, # MongoDB: Alta, Baja
), nrow = 2, byrow = TRUE)
```

```
# Definir las probabilidades de los estados de la naturaleza
probabilidades <- c(Demanda_Alta = 0.6, Demanda_Baja = 0.4)

# Nombrar filas y columnas para claridad
rownames(resultados) <- c("PostgreSQL", "MongoDB")
colnames(resultados) <- names(probabilidades)

print(resultados)
```

Paso 2: Calcular el Valor Monetario Esperado (VME)

El VME se calcula como: $VME = \sum (Resultado_i \times Probabilidad_i)$

```
R
# Multiplicar la matriz de resultados por el vector de probabilidades
VME_PostgreSQL <- sum(resultados["PostgreSQL", ] * probabilidades)
VME_MongoDB <- sum(resultados["MongoDB", ] * probabilidades)

# Crear un dataframe para comparar
VMEs <- data.frame(
  Alternativa = c("PostgreSQL", "MongoDB"),
  VME = c(VME_PostgreSQL, VME_MongoDB)
)

print(VMEs)

# Decisión
decisión_1 <- VMEs[which.max(VMEs$VME), ]
cat("\nLa decisión óptima es:", decisión_1$Alternativa,
    "con un VME de", round(decisión_1$VME, 2), "mil USD.")
```

2. Ejemplo: Decisión sobre Inversión en Ciberseguridad

Escenario

Una startup debe decidir si invierte **Moderado** o **Alto** en su sistema de defensa contra ataques. El resultado (costo/pérdida) depende de si ocurre un **Ataque Masivo (AM)** o un **Ataque Menor (Am)**. Como se trata de costos, el objetivo es **minimizar el Costo Esperado (CE)**.

Matriz de Resultados (Costo Total, en miles de USD)

Alternativa	Ataque Masivo (Prob=0.1)	Ataque Menor (Prob=0.9)
A1: Inversión Moderada	500	50
A2: Inversión Alta	150	80

Exportar a Hojas de cálculo

Análisis de Decisiones con RStudio

Paso 1: Definir la Matriz y Probabilidades

```
# Definir los costos (matriz de pérdidas)
costos <- matrix(c(
  500, 50, # Moderada: AM, Am
  150, 80  # Alta: AM, Am
), nrow = 2, byrow = TRUE)

probabilidades_ataque <- c(Ataque_Masivo = 0.1, Ataque_Menor = 0.9)

rownames(costos) <- c("Inversion_Moderada", "Inversion_Alta")
colnames(costos) <- names(probabilidades_ataque)

print(costos)
```

Paso 2: Calcular el Costo Esperado (CE)

```
R
# Calcular el Costo Esperado (CE) para cada alternativa
CE_Moderada <- sum(costos["Inversion_Moderada", ] *
  probabilidades_ataque)
CE_Alta <- sum(costos["Inversion_Alta", ] * probabilidades_ataque)

CEs <- data.frame(
  Alternativa = c("Inversion_Moderada", "Inversion_Alta"),
  CE = c(CE_Moderada, CE_Alta)
)

print(CEs)

# Decisión: Minimizar el CE
decisión_2 <- CEs[which.min(CEs$CE), ]
cat("\nLa decisión óptima es:", decisión_2$Alternativa,
    "con un Costo Esperado de", round(decisión_2$CE, 2), "mil USD.")
```

3. Ejemplo: Decisión sobre Contratación en Proyecto

Escenario

El gerente de un proyecto debe decidir si **Contrata un Experto** externo para una tarea crítica o si deja que el **Equipo Actual** la desarrolle. El resultado (costo por retrasos) depende de si la **Integración** resulta ser **Fácil** o **Difícil**.

Matriz de Resultados (Costo por Retraso, en días-hombre)

Alternativa	Integración Fácil (Prob=0.7)	Integración Difícil (Prob=0.3)
A1: Equipo Actual	10	120
A2: Contratar Experto	40	50

Análisis de Decisiones con RStudio

Paso 1: Definir la Matriz y Probabilidades

```
costos_retraso <- matrix(c(
  10, 120, # Equipo Actual: Fácil, Difícil
  40, 50   # Experto: Fácil, Difícil
), nrow = 2, byrow = TRUE)

probabilidades_integracion <- c(Facil = 0.7, Difícil = 0.3)

rownames(costos_retraso) <- c("Equipo_Actual", "Contratar_Experto")
colnames(costos_retraso) <- names(probabilidades_integracion)

print(costos_retraso)
```

Paso 2: Calcular el Costo Esperado (CE)

```
CE_Actual <- sum(costos_retraso["Equipo_Actual", ], probabilidades_integracion) *
CE_Experto <- sum(costos_retraso["Contratar_Experto", ], probabilidades_integracion) *

CEs_3 <- data.frame(
  Alternativa = c("Equipo_Actual", "Contratar_Experto"),
  CE = c(CE_Actual, CE_Experto)
)

print(CEs_3)

# Decisión: Minimizar el CE
decisión_3 <- CEs_3[which.min(CEs_3$CE), ]
cat("\nLa decisión óptima es:", decisión_3$Alternativa,
    "con un Costo Esperado (días-hombre) de", round(decisión_3$CE, 2),
    ".")
```

4. Ejemplo: Decisión sobre Actualización de Servidor

Escenario

El administrador de sistemas debe decidir si **Actualiza el Servidor (AS)** antes del evento de alto tráfico o si **Mantiene el Servidor Actual (MA)**. El resultado (pérdida de beneficios) depende de si el **Tráfico es Alto** o **Bajo**.

Matriz de Resultados (Pérdida de Beneficios, en miles de USD)

Alternativa	Tráfico Alto (Prob=0.25)	Tráfico Bajo (Prob=0.75)
A1: Mantener Actual	60	5
A2: Actualizar Servidor	10	15

Análisis de Decisiones con RStudio

Paso 1: Definir la Matriz y Probabilidades

```
R
perdidas <- matrix(c(
  60, 5,   # Mantener Actual: Alto, Bajo
  10, 15  # Actualizar: Alto, Bajo
), nrow = 2, byrow = TRUE)

probabilidades_trafico <- c(Alto = 0.25, Bajo = 0.75)

rownames(perdidas) <- c("Mantener_Actual", "Actualizar_Servidor")
colnames(perdidas) <- names(probabilidades_trafico)

print(perdidas)
```

Paso 2: Calcular el Costo Esperado (CE)

```
R
CE_Mantener <- sum(perdidas["Mantener_Actual", ] *
probabilidades_trafico)
CE_Actualizar <- sum(perdidas["Actualizar_Servidor", ] *
probabilidades_trafico)

CEs_4 <- data.frame(
  Alternativa = c("Mantener_Actual", "Actualizar_Servidor"),
  CE = c(CE_Mantener, CE_Actualizar)
)

print(CEs_4)

# Decisión: Minimizar el CE
decisión_4 <- CEs_4[which.min(CEs_4$CE), ]
cat("\nLa decisión óptima es:", decisión_4$Alternativa,
    "con un Costo Esperado de", round(decisión_4$CE, 2), "mil USD.")
```

Ejercicios Propuestos de Análisis de Decisiones con RStudio

Ejercicio 1: Desarrollo de Módulo vs. Compra de Licencia (VME)

Una empresa de software debe decidir cómo obtener un módulo de funcionalidad crítica: **Desarrollarlo Internamente** o **Comprar una Licencia**. El resultado (beneficio, en miles de USD) depende de la **Aceptación del Cliente** (éxito en la venta del producto). El objetivo es maximizar el VME.

Alternativa	Aceptación Alta (Prob=0.7)	Aceptación Baja (Prob=0.3)
A1: Desarrollar Internamente	180	50
A2: Comprar Licencia	140	80

Tarea en RStudio:

1. Calcular el VME para cada alternativa.
2. Determinar la decisión óptima.

Ejercicio 2: Estrategia de Prueba de Software (CE)

Un equipo de QA debe decidir si realiza **Pruebas Exhaustivas** (costo inicial alto, menos fallos) o **Pruebas Rápidas** (costo inicial bajo, más fallos potenciales). El resultado es el costo total (inicial + costo esperado por *bugs* post-lanzamiento, en USD). El objetivo es minimizar el Costo Esperado (CE).

Alternativa	Fallo Mayor (Prob=0.1)	No Hay Fallo Mayor (Prob=0.9)
A1: Pruebas Exhaustivas	12,000	4,000
A2: Pruebas Rápidas	25,000	2,000

Tarea en RStudio:

1. Calcular el CE para cada alternativa.
2. Determinar la decisión óptima.

Ejercicio 3: Elección de Servidor Cloud (VME)

Una aplicación web tiene dos opciones de servidor *cloud* para el lanzamiento: **Servidor Básico (B)** o **Servidor Premium (P)**. El resultado (ganancia, en USD) depende de si la **Carga de Usuarios** es **Crítica** o **Normal**.

Alternativa	Carga Crítica (Prob=0.3)	Carga Normal (Prob=0.7)
A1: Servidor Básico (B)	500	1,500
A2: Servidor Premium (P)	1,800	1,000

Tarea en RStudio:

1. Calcular el VME para cada alternativa.
2. Determinar la decisión óptima.

Ejercicio 4: Inversión en Seguridad de Datos (CE)

Un departamento de TI debe decidir si **Aplica un Parche Inmediato** (costo de tiempo hombre) o si **Espera la Actualización Trimestral** (costo bajo, pero riesgo de ataque). El resultado es el costo total (en horas-hombre perdidas). El objetivo es minimizar el CE.

Alternativa	Explotación de Vulnerabilidad (Prob=0.15)	No Hay Explotación (Prob=0.85)
A1: Aplicar Parche Inmediato	20	10
A2: Esperar Actualización	150	5

Tarea en RStudio:

1. Calcular el CE para cada alternativa.
2. Determinar la decisión óptima.

Ejercicio 5: Estrategia de Implementación Ágil (VME)

El director de proyecto debe elegir entre dos estrategias de implementación: **Lanzamiento Múltiple (LM)** o **Lanzamiento Único Grande (LUG)**. El resultado (puntuación de satisfacción del cliente) depende de la **Estabilidad del Código** (alta o baja). El objetivo es maximizar el VME (Satisfacción).

Alternativa	Estabilidad Alta (Prob=0.4)	Estabilidad Baja (Prob=0.6)
A1: Lanzamiento Múltiple (LM)	95	70
A2: Lanzamiento Único Grande (LUG)	100	55

Tarea en RStudio:

1. Calcular el VME para cada alternativa.
2. Determinar la decisión óptima.

Plantilla de Código Única para RStudio

Puedes usar esta plantilla de código y simplemente cambiar las variables resultados y probabilidades para resolver cualquiera de los 5 ejercicios.

R

```
# --- PLANTILLA DE ANÁLISIS DE DECISIONES CON VME/CE ---

# PASO 1: Ingresar los datos del ejercicio

# EJEMPLO: Usando los datos del Ejercicio 1
resultados <- matrix(c(
  180, 50, # A1: Desarrollar
  140, 80  # A2: Licencia
), nrow = 2, byrow = TRUE)

probabilidades <- c(P1 = 0.7, P2 = 0.3)

# Nombrar para claridad (Asegúrate de cambiar los nombres de fila y
columna según el ejercicio)
rownames(resultados) <- c("Alternativa_1", "Alternativa_2")
colnames(resultados) <- names(probabilidades)

# PASO 2: Calcular el Valor Esperado (VME o CE)

# Multiplicar la matriz por el vector de probabilidades
VME_A1 <- sum(resultados["Alternativa_1", ] * probabilidades)
VME_A2 <- sum(resultados["Alternativa_2", ] * probabilidades)

# Crear un dataframe de resultados
VMEs <- data.frame(
  Alternativa = rownames(resultados),
  Valor_Esperado = c(VME_A1, VME_A2)
)

print("--- Matriz de Resultados ---")
print(resultados)
print("--- Valores Esperados ---")
print(VMEs)

# PASO 3: Tomar la Decisión Óptima
# Si el resultado es GANANCIA (VME), se busca el máximo (which.max)
# Si el resultado es COSTO (CE), se busca el mínimo (which.min)

# En este caso (Ej. 1), buscamos MAXIMIZAR la ganancia
decisión_final <- VMEs[which.max(VMEs$Valor_Esperado), ]

cat("\nDecisión Óptima:", decisión_final$Alternativa,
    "con un VME de", round(decisión_final$Valor_Esperado, 2))

# Si el ejercicio fuera de COSTO (CE), la línea de decisión cambiaría
a:
# decisión_final <- VMEs[which.min(VMEs$Valor_Esperado), ]
```